

CARGOFLOW SENTINEL

A three-agent watch system that catches shipment problems before they turn into a missed delivery, a spoiled load, or a stalled production line.

[3-Agent System](#)[Built on Microsoft Foundry](#)[Logistics / Supply Chain](#)[2026 Microsoft Virtual Agent-a-thon](#)github.com/ssfathimaIndia-wq/cargoflow-sentinel ↗

01 — THE PROBLEM

SHIPMENTS GET WATCHED BY WHOEVER REMEMBERS TO LOOK.

Coordinators track dozens of shipments against SLAs — delivery windows, customs timing, cold-chain temperature limits — mostly through periodic manual review of dashboards and spreadsheets.

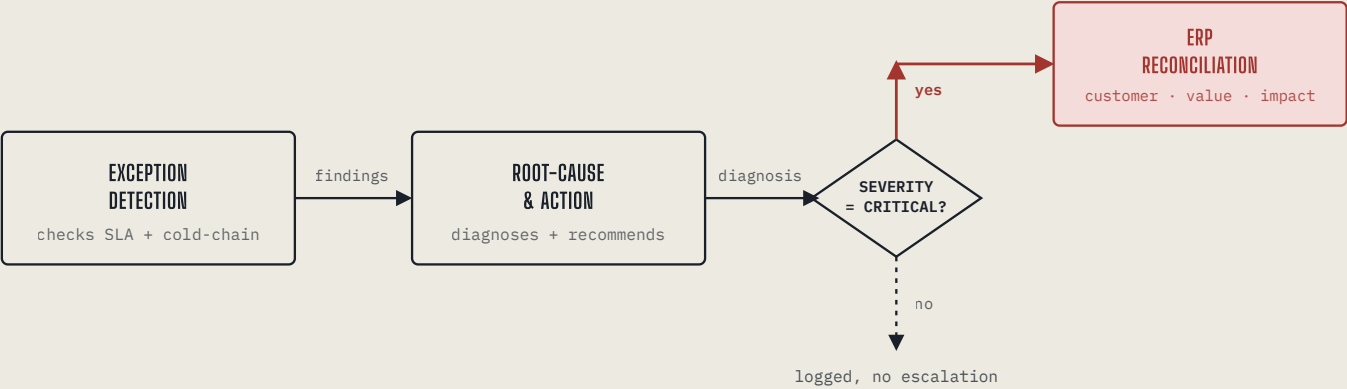
That approach has three weaknesses: it doesn't scale past a handful of shipments, it's inconsistent from one coordinator to the next, and it's reactive — by the time someone actually notices a problem, the shipment has often already blown its window. A refrigeration fault that's been running for six hours costs the same whether it's caught at hour one or hour six; the difference is entirely in how fast someone looked.

The expensive part isn't the delay itself. It's what the delay touches: a spoiled cold-chain load, a demurrage fee, or — when the shipment feeds a factory — a production line waiting on a part that isn't coming.

02 — THE SOLUTION

THREE AGENTS, ONE OF THEM CONDITIONAL.

CargoFlow Sentinel is a chain of three purpose-built agents. The first two run on every flagged shipment. The third only runs when the stakes are actually high — which is the part that makes this more than a linear alert pipeline.



The first two agents run for every flagged shipment. The Root-Cause & Action Agent's diagnosis feeds a single branch: only CRITICAL severity continues to the ERP Reconciliation Agent, which is the piece that turns a technical exception into a business-impact statement.

Here's the chain acting on one real shipment from the system's own test data:

SHP-1003

● CRITICAL

ROUTE

Gdansk → Berlin

EXCEPTION

customs delay +12.8 hrs

LINKED PO

PO-88203 · NordCold Distribution

CARRIER

BalticRoute GmbH

COLD-CHAIN

9.4°C vs 6.0°C limit (+56.7%)

ORDER VALUE

€41,200 · production-dependent

Because this one crosses into CRITICAL on two fronts, the ERP Reconciliation Agent fires automatically and reports back: a €41,200 order at a customer whose production line depends on it, flagged for urgent escalation — not just "temperature exceeded."

DESIGN, BUILD, RUN BOTH PATHS, MEASURE.

Built on Microsoft Foundry, extending the pattern from Microsoft's own official hackathon lab — a two-agent inspection pipeline — with a third agent and the conditional branch above.

1

Design against real entities

The purchase-order lookup is modeled on real ERP fields — customer account, order value, production dependency — the same shape used in Microsoft Dynamics 365 Finance & Operations, not invented placeholders.

2

Build and deploy live

Three agents deployed to a real Foundry project — not a design mockup. Each has its own instructions and, where needed, its own tool: a shipment-health check and a purchase-order lookup.

3

Orchestrate two ways

A Python service calls the three agents directly through the SDK — the production-grade path. A visual workflow built in Foundry's own drag-and-drop builder chains the same three agents with the same branch, as a second, independently verified path.

4

Evaluate, don't just demo

Every diagnosis is scored by a separate judging model against hand-written test cases with known-correct answers, not eyeballed once and called done.

04 — INDUSTRY & MARKET FIT

LOGISTICS FIRST, WITH AN ERP HAND-OFF.

The core fit is **freight & logistics** — anyone moving goods across a customs border or a temperature threshold: cold-chain food and pharma shippers, contract manufacturers, freight forwarders. The ERP Reconciliation Agent gives it a second

fit in **finance/ERP integration**, specifically Microsoft Dynamics 365 Finance & Operations shops running purchase orders against inbound freight.

ALREADY WELL-ESTABLISHED

- Shipment exception monitoring by severity — "control tower" software is a known category
- Automated root-cause suggestions for logistics delays
- Cold-chain temperature alerting

WHERE THIS DIFFERS

- No official scenario in this contest's own lab library (predictive maintenance, insurance claims, financial compliance) covers logistics
- The conditional ERP hop — escalation gated on real business impact, not just severity — isn't in that library either
- Grounded in real D365FO/CargoWise integration work, not a generic demo dataset

Stated plainly: this doesn't claim to invent shipment monitoring. It extends a known pattern into a domain this specific contest's materials don't cover, built on real integration experience.

05 — BUSINESS VALUE

THE VALUE IS IN THE TIMING, NOT THE AUTOMATION.

Every value category below is a real, named cost that earlier detection avoids — no invented savings figures, because this is a working prototype on representative data, not a production deployment with a year of measured outcomes yet.

AVOIDED DEMURRAGE A customs hold caught at hour one instead of hour twelve is a phone call, not a fee.	AVOIDED SPOILAGE A refrigeration fault flagged while the truck is still moving is a redirected load, not a written-off one.
PROTECTED PRODUCTION LINES When a shipment feeds a factory, the ERP agent surfaces that dependency	AUTOMATIC TRIAGE A coordinator watching twenty shipments doesn't have to manually cross-reference

automatically — before the line stalls, not after.

which CRITICAL case is also the €58,900 one.

06 — HOW WE EVALUATE IT

SCORED BY A JUDGE, NOT JUST WATCHED ONCE.

A second model grades every diagnosis against hand-written test cases with defined expectations — did it name the right root cause, was the recommended action specific enough to act on immediately, did it avoid over-escalating a WARNING into CRITICAL language.

4.5_{/5}

AVG. QUALITY SCORE

SHP-1002		5/5
SHP-1003		5/5
SHP-1005		5/5
SHP-1005 · ERP		3/5

"Correctly highlights production impact and urgent escalation, but lacks specific, logistics-actionable next steps" — the judge model's own note on the one lower score, kept as-is rather than re-run until it looked better.

That score sits alongside functional proof, not instead of it: both orchestration paths — the Python service and the Foundry visual workflow — run this exact chain end-to-end against the live deployed agents, branch correctly on severity, and produce this same kind of output.

Innovation

A conditional, business-impact-aware third hop — not just another linear alert pipeline.

Usability

Judged output plus two independently working execution paths, not a single lucky run.

Impact

Named, specific value categories tied to real ERP fields — not abstract "AI efficiency."

TWO THINGS WORTH STATING PLAINLY, NOT GLOSSING OVER.

"Production-ready" claims are easy to overstate. Here's exactly what's real for these two, and what isn't yet.

RETRIES — BUILT Every agent call is wrapped in exponential-backoff retry logic, but only for genuinely transient failures — rate limits, connection drops, timeouts, 5xx errors. Auth and bad-request errors are never retried, since retrying those just wastes time on an error that fails identically every time. Verified with a full live workflow run and a full eval run, both clean after the change.	PRIVACY — ACKNOWLEDGED, NOT BUILT The current dataset is synthetic shipment and purchase-order data with no real personal information, so this isn't an issue today. A production connection to live carrier and D365FO data would need PII handling appropriate to GDPR/EMEA data residency requirements before going live — a known next step, stated here rather than left unsaid.
---	---

WHAT'S ACTUALLY RUNNING, AND WHAT ISN'T YET.

Worth separating clearly, because it changes what you should believe about this system.

PIECE	STATUS
● Three live agents on Foundry	Deployed and running, not a mockup.
● Conditional ERP branch	Verified working on two independent orchestration paths.
● Judged evaluation	4.5/5 across hand-written cases, including the honest low score.
● Retry handling	Exponential backoff on transient API errors only; auth/bad-request errors never retried.

PIECE**STATUS**

- | PIECE | STATUS |
|--|---|
| ● Data sources | Structured mock data shaped like the real thing — not yet a live carrier-tracking feed or a live D365FO connection. |
| ● Standing deployment | Runs on demand today. A production version needs a schedule or event trigger, not a terminal command. |
| ● Privacy / PII handling | Not needed for synthetic data today; required before connecting real carrier/D365FO data. |
| ● Foundry's visual workflow builder | Useful for this proof-of-concept; Microsoft is retiring it Dec 1, 2026. The Python path is the durable one. |

CargoFlow Sentinel — 2026 Microsoft Virtual Agent-a-thon, EMEA region · Built on Microsoft Foundry ·
Code: github.com/ssfathimaIndia-wq/cargoflow-sentinel